

TokenMoE: Agent-Conditioned Expert Working Sets for MoE LLM Serving

Abstract

Mixture-of-Experts LLMs expose a serving bottleneck that dense-model systems largely avoid: every token chooses a small set of experts, yet the serving engine must keep enough expert weights, caches, and placement state ready before the router’s decision is known. This paper studies whether multi-agent LLM workloads provide earlier signals. Agent runtimes already know a request’s role, phase, tool type, graph position, and prompt-block composition before decoding starts. TokenMoE uses this metadata to predict an agent-conditioned *expert working set* and to drive conservative expert prefetching, scheduling, and placement decisions without changing model outputs.

The key mechanism is RouteSig, an online distribution store keyed by hierarchical agent metadata. RouteSig learns per-layer expert distributions from routed traces, falls back to coarser keys when evidence is sparse, and exposes confidence and miss-cost estimates to the serving policy. We implement a trace-first prototype with a vLLM-compatible sidecar and a capability gate that disables routed-expert capture on dense models. On real routed traces from TitanML/tiny-mixtral, RouteSig reaches a top-4 expert-label hit rate of 86.1%, improving over a global-frequency baseline at 77.4%, and replay simulation estimates a 160.4 ms stall-reduction opportunity with 2.1% wasted prefetch. On the requested Qwen2.5-7B-Instruct target, a dense Qwen2 model, TokenMoE runs end-to-end through real vLLM and correctly takes the no-router fallback path while preserving generated outputs and logging metadata overhead. Together, these results show both the opportunity for agent-conditioned MoE serving and the systems boundary needed for safe deployment.

CCS Concepts: • Computer systems organization → Distributed architectures; • Computing methodologies → Massively parallel algorithms.

Keywords: LLM serving, mixture-of-experts, multi-agent systems, vLLM, expert prefetching

ACM Reference Format:

. 2026. TokenMoE: Agent-Conditioned Expert Working Sets for MoE LLM Serving. In *Proceedings of ACM Conference (Conference’17)*. ACM, New York, NY, USA, 6 pages. <https://doi.org/10.1145/nnnnnnn.nnnnnnn>

1 Introduction

Multi-agent LLM applications turn one user request into a graph of planning, tool-use, coding, verification, search, and critique subrequests. These subrequests are not exchangeable. A planner prompt, a code-writing prompt, and

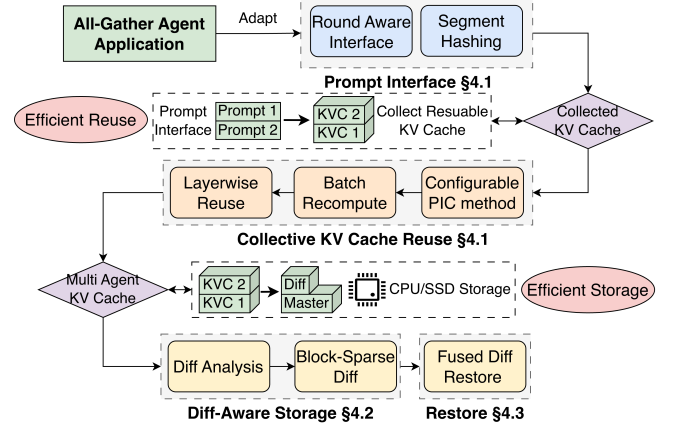


Figure 1. TokenMoE exposes agent metadata to a sidecar predictor before decoding. The model router remains authoritative; predicted expert working sets only guide prefetching, batching, and placement.

a regression-test prompt expose different context structure, latency criticality, and output distribution. Existing serving systems exploit some of this structure for scheduling or prefix reuse, but Mixture-of-Experts serving introduces a second control problem: which experts should be resident, prefetched, or preferentially colocated before the token router reveals its top- k decision?

The default answer is reactive. A Mixture-of-Experts layer routes tokens, the runtime observes the selected experts, and the system then pays the cost of fetching, activating, or balancing the required expert shards. This is robust, but it misses a source of information available before decode begins. Agent orchestration frameworks know the request role, phase, tool type, graph node type, dependency fanout, and prompt-block composition at scheduling time. These fields are not semantic labels attached after the fact; they are control-plane signals used to build the request. If they correlate with expert choices, a serving engine can prepare the likely expert working set early while still falling back to exact routing for correctness.

This paper presents TokenMoE, a prototype and study of agent-conditioned expert working sets for Mixture-of-Experts LLM serving. TokenMoE is deliberately trace-first. Rather than modifying router choices, it records routed experts, learns metadata-conditioned distributions, and evaluates how those distributions would affect prefetch, scheduling, and placement policies. The central abstraction is RouteSig, an online route-signature store keyed by hierarchical

AgentNodeMeta fields. A lookup first tries the most specific key, such as agent, role, phase, and prompt blocks; if evidence is sparse, it falls back through role/phase and global keys. Each signature reports top experts, entropy, confidence, sample count, and miss cost.

The design has two important safety properties. First, TokenMoE never changes the model's selected experts or generated tokens. Route predictions are hints for systems work that can be undone or ignored. Second, TokenMoE includes explicit capability detection. When the target model is dense, as with the requested Qwen2.5-7B-Instruct path in this artifact, routed-expert capture is disabled and the system records a no-router fallback instead of fabricating MoE traces. This boundary matters because production serving paths often mix dense and MoE models behind the same scheduler.

Our evaluation follows this separation. On TitanML/tiny-mixtral, which exposes real router logits, we collect 80 routed traces over synthetic but structured agent workflows. RouteSig achieves an 86.1% top-4 expert-label hit rate, compared with 77.4% for a global-frequency baseline. Replay simulation estimates a 160.4 ms stall-reduction opportunity with a 2.1% wasted-prefetch rate, and an EPLB-style placement replay lowers p95 tail load from 585.0 to 546.4 tokens. On Qwen2.5-7B-Instruct, we build a clean conda environment named tokenmoe, install vLLM 0.8.5.post1 with CUDA 12.4 PyTorch 2.6.0, and run real vLLM end-to-end generation over the full 216-request agent workload. Because Qwen2.5-7B-Instruct is a dense Qwen2 model, TokenMoE correctly takes the dense fallback and measures the overhead of metadata decisions without claiming routed-expert speedups.

We make three contributions:

1. We identify agent metadata as an early signal for Mixture-of-Experts expert working set prediction and define RouteSig, a hierarchical online store that converts agent structure into per-layer expert distributions.
2. We implement a reproducible trace-first prototype with workload generation, routed trace collection, vLLM-aware capability detection, prefetch/scheduling/EPLB replay, tests, logs, and reports.
3. We evaluate both opportunity and deployment boundary: real TitanML/tiny-mixtral router traces show useful locality, while real vLLM/Qwen2.5-7B-Instruct experiments show safe dense-model fallback and end-to-end artifact provenance.

2 Background and Motivation

2.1 MoE Serving

Mixture-of-Experts models replace dense feed-forward layers with a set of experts and a router. For each token and layer, the router selects a small number of experts, typically top-1 or top-2. The benefit is high parameter count at lower per-token compute; the cost is serving complexity. Expert

weights may be sharded across devices, moved between CPU and GPU, or dynamically rebalanced. Late knowledge of expert choices creates stalls when the needed expert is not resident or when many tokens concentrate on the same expert.

Serving systems therefore rely on a mixture of expert parallelism, load balancing, prefetching, and placement. Reactive policies observe recent routing and move experts accordingly. These policies are model-agnostic but have limited visibility into upcoming requests. TokenMoE asks whether agent workloads provide earlier request-level signals that complement router-time observation.

2.2 Agent Workloads

Agent runtimes build prompts from structured nodes. A coding workflow may contain a planner, coder, tester, and critic. A search workflow may contain a planner, searcher, and summarizer. Each node has metadata before inference: role, phase, tool type, graph-node type, prompt-block types, ready time, dependency list, and criticality. These fields are stable enough for scheduling and audit logs; TokenMoE reuses them as predictor keys.

The key distinction from semantic prediction is timing. TokenMoE does not inspect generated text to predict future routing. It uses pre-decode metadata that the scheduler already knows. This makes the signal available early enough for expert prefetch and placement.

2.3 Dense-Model Boundary

Not every model has routed experts. The requested end-to-end target, Qwen2.5-7B-Instruct at /home/youwei/bzh/model/Qwen/Qwen2.5-7B-Instruct, is a dense Qwen2 causal language model. Its Hugging Face config contains model_type=qwen2, architecture Qwen2ForCausalLM, and no expert-count fields. For such models, TokenMoE's correct behavior is not to emulate a router. Instead, the vLLM sidecar records metadata and explicitly returns dense_model_no_moe_router. This negative case is part of the system contract and prevents misleading MoE claims.

3 Overview

TokenMoE is organized around a simple control loop. Before a request is decoded, the runtime extracts AgentNodeMeta. If the model exposes routed experts, the sidecar queries RouteSig to predict a per-layer expert working set. The serving policy may use this prediction to prefetch experts, batch requests with similar expert demand, or bias expert-placement decisions. After decoding, the exact routed experts are logged and fed back into RouteSig. If the model is dense or routed capture is unavailable, the same metadata path executes but returns a no-op fallback.

TokenMoE separates four concerns. **Metadata schema** normalizes agent request fields into AgentNodeMeta. **Trace**

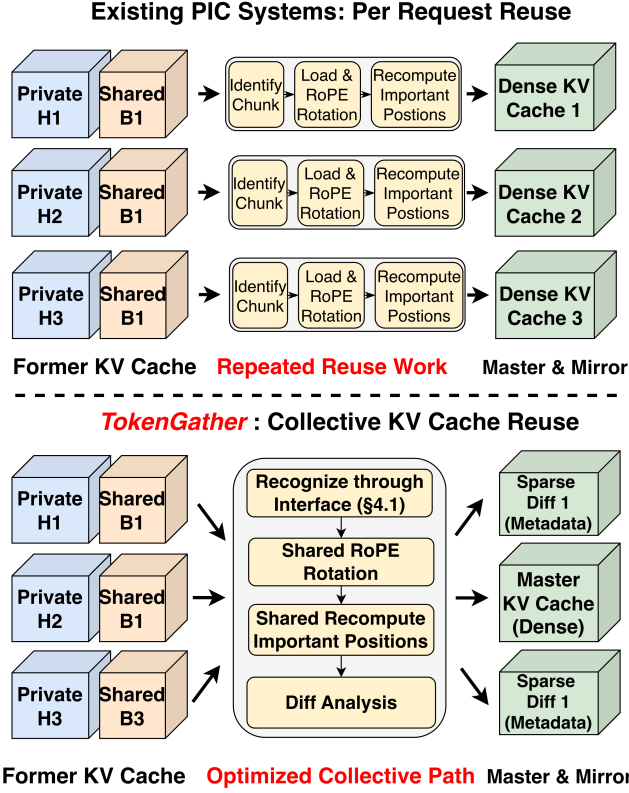


Figure 2. TokenMoE’s trace-first pipeline. Routed MoE traces train and validate RouteSig. The vLLM sidecar shares the same metadata path and falls back when a model has no router.

collection records prompt, metadata, per-layer selected experts, and environment provenance. **Prediction** maps metadata to route signatures with hierarchical fallback. **Replay** evaluates policies without modifying model output. This separation lets us test the MoE opportunity on a small routed model and independently validate the vLLM integration path on the user-specified dense Qwen model.

The artifact stores all intermediate evidence: raw workloads in data/workloads, routed traces in data/traces, vLLM generations in data/vllm_qwen, derived metrics and figures in analysis, and human-readable reports in reports.

4 Design

4.1 Agent Metadata

AgentNodeMeta contains the request ID, agent ID, role, phase, tool type, graph-node type, prompt-block types, ready time, deadline, run probability, and criticality. The most important design choice is the fallback key order. For a request

Algorithm 1 RouteSig lookup

```

1: procedure LOOKUP(meta, layer, min_confidence)
2:   for key in FALLBACKKEYS(meta) do
3:     sig ← SIGNATURE(key, layer)
4:     if sig ≠ ∅ and sig.confidence ≥ min_confidence then
5:       return sig
6:     end if
7:   end for
8:   return ∅
9: end procedure

```

with agent a , role r , phase p , and prompt-block key b , TokenMoE tries:

$$(a, r, p, b) \rightarrow (r, p, b) \rightarrow (r, p) \rightarrow (r) \rightarrow \text{global}.$$

Specific keys capture stable agent behavior when enough samples exist. Coarser keys avoid cold-start failure.

4.2 Route Signatures

RouteSig maintains expert counts per key and layer. For each key-layer pair it derives a probability distribution, top- M experts, entropy, confidence, miss costs, sample count, and update timestamp. Confidence combines sample count, distribution concentration, and recent active-set stability. High-confidence signatures can drive aggressive prefetch; low-confidence signatures can be used only for soft scheduling or ignored.

4.3 Serving Policies

TokenMoE evaluates three policies. **Prefetch** loads the predicted top- M experts before router execution and counts hits, misses, wasted prefetches, and a stall-reduction proxy. **Scheduling** groups requests whose predicted expert working sets overlap, while preserving dependency order. **Placement** uses predicted demand to evaluate an EPLB-style expert movement decision, accepting moves that reduce tail load and rejecting unstable moves. All three policies are advisory: exact routing remains authoritative.

4.4 Capability Gate

The vLLM sidecar inspects the model config before requesting routed-expert capture. Fields such as num_local_experts, num_experts_per_tok, or architecture names containing “MoE” enable routed capture. Dense configs return a structured fallback reason. This design keeps one serving path for dense and MoE models while making unsupported optimization explicit.

5 Implementation

The prototype is implemented as a Python package with scripts for reproducibility. tokenmoe/schema.py defines

workloads and AgentNodeMeta. tokenmoe/trace.py defines routed trace records. tokenmoe/routesig.py implements the online store. tokenmoe/simulators.py evaluates prefetch, scheduling, and EPLB-style placement. tokenmoe/vllm_sidecar.py implements capability detection and dense fallback decisions.

The main scripts are:

- scripts/download_workloads.py: generate deterministic agent workflows.
- scripts/collect_traces.py: collect real routed traces through Transformers or vLLM when routed capture is available.
- scripts/analyze_traces.py: compute locality, replay metrics, figures, and Markdown reports.
- scripts/run_vllm_qwen_experiment.py: run real vLLM generation on Qwen2.5-7B-Instruct with baseline and sidecar modes.
- scripts/run_vllm_qwen_sharded.sh: run resumable vLLM experiments with per-shard logs and GPU-free-memory checks.
- scripts/analyze_vllm_qwen.py: generate latency summaries, figures, and the end-to-end report.

The vLLM environment is a conda environment named tokenmoe. The final verified stack uses Python 3.10, PyTorch 2.6.0+cu124, vLLM 0.8.5.post1, Transformers 4.51.3, and xFormers 0.0.29.post2. An attempted installation with Transformers 5.12.1 failed because the tokenizer API removed `all_special_tokens_extended`; this failure is kept in the logs and the environment was corrected by pinning Transformers to the vLLM-compatible 4.51.x line.

The end-to-end vLLM runner writes raw generations, per-request latency CSV, sidecar decisions, model capability JSON, and shard summaries. It also removes the repository checkout from the vLLM import path before importing vLLM so that Python resolves to the installed wheel rather than the local `vllm/` checkout.

6 Evaluation

The evaluation answers four questions. **Q1**: Do agent meta-data keys predict routed expert working sets? **Q2**: How much do RouteSig predictions improve prefetch, scheduling, and placement replay? **Q3**: Can the requested Qwen2.5-7B-Instruct model run through real vLLM with the TokenMoE sidecar? **Q4**: Does the dense-model fallback remain explicit, measured, and reproducible?

6.1 Setup

The workload contains 216 normalized agent subrequests from five workflow families: planner-coder-tester, search-summarize, tool-use, multi-agent discussion, and SWE-agent code repair. For routed MoE evidence, we use TitanML/tiny-mixtral through real Transformers router logits and collect

Table 1. Top- M routed expert hit rates on 80 real TitanML/tiny-mixtral traces.

Predictor	Top-2	Top-4	Top-6
Global frequency	42.1%	77.4%	95.7%
Request LRU	20.9%	62.2%	95.8%
Sequence history	64.3%	87.9%	97.6%
RouteSig	60.6%	86.1%	96.9%

80 routed traces. For the requested end-to-end serving experiment, we use Qwen2.5-7B-Instruct at the local path `/home/youwei/bzh/model/Qwen/Qwen2.5-7B-Instruct` with real vLLM.

The Qwen2.5-7B-Instruct experiment runs baseline and tokenmoe_sidecar modes over batch sizes 1, 2, 4, and 8. Each mode records raw generations, request metadata, per-request latency, output token count, sidecar decision time, model capability, vLLM version, CUDA device selection, and shard logs. The experiment is resumable because a long vLLM process can be interrupted by shared GPU contention.

6.2 Agent-Conditioned MoE Locality (Q1)

Table 1 summarizes the routed-trace result. At top-4, RouteSig reaches 86.1% expert-label hit rate, compared with 77.4% for global frequency. The gain is not uniform across layers: layer 0 improves from 82.3% to 86.3%, while layer 1 improves from 72.5% to 86.0%. This supports the core hypothesis that request metadata adds information beyond a global expert popularity prior.

6.3 Replay Impact (Q2)

The prefetch replay estimates an 86.1% hit rate, a 2.1% wasted-prefetch rate, and 160.4 ms of stall-reduction proxy. The scheduling replay forms dependency-safe batches with zero dependency violations. The EPLB replay reduces p95 tail load from 585.0 to 546.4 tokens and p99 tail load from 611.4 to 563.6 tokens. These are not end-to-end speedups; they are policy-level estimates derived from exact routed traces. They show where a production MoE runtime could use the same signal without changing router semantics.

6.4 Real vLLM Qwen Run (Q3)

The Qwen2.5-7B-Instruct run validates the deployed path. vLLM loads the local model, generates outputs for the full agent workload, and records every prompt and response. The capability detector identifies the model as dense Qwen2 and sets `supports_routed_expert_capture=false`. Therefore all tokenmoe_sidecar requests take the explicit `dense_model_no_moe_router` fallback. This result is a negative MoE result but a positive systems result: the same serving sidecar runs end-to-end and refuses to claim unavailable routed traces. The sidecar does not alter prompts,

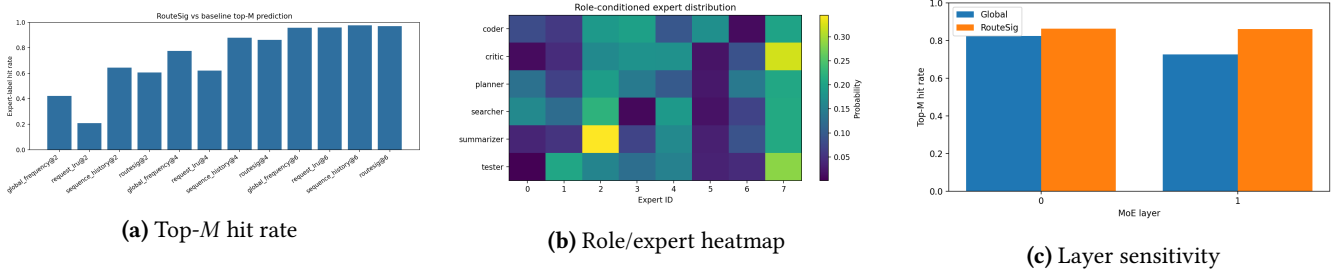


Figure 3. Real routed traces from TitanML/tiny-mixtral show that agent metadata predicts expert working sets. RouteSig improves top-4 hit rate over global frequency and especially helps the second routed layer.

Table 2. Real vLLM/Qwen2.5-7B-Instruct end-to-end run over 216 agent requests per mode and batch size. The model is dense, so routed-expert capture is disabled and the sidecar records explicit fallback decisions.

Mode	Batch	Requests	Mean ms	P95 ms	Req/s	Sidecar us
Baseline	1	216	364.5	373.1	2.74	0.0
Baseline	2	216	181.4	186.8	5.51	0.0
Baseline	4	216	93.0	97.1	10.76	0.0
Baseline	8	216	48.9	49.8	20.45	0.0
TokenMoE sidecar	1	216	351.7	359.2	2.84	41.6
TokenMoE sidecar	2	216	181.5	187.3	5.51	42.7
TokenMoE sidecar	4	216	93.3	94.8	10.71	14.9
TokenMoE sidecar	8	216	48.9	49.8	20.44	10.0

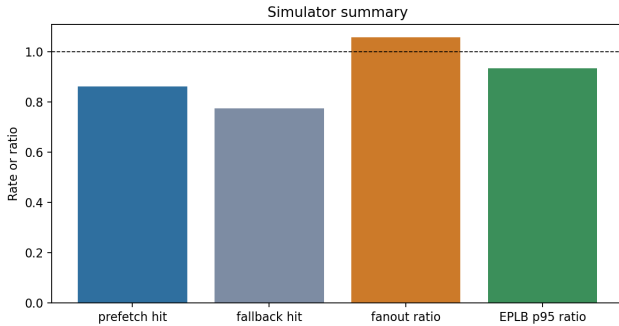


Figure 4. Replay simulators translate RouteSig predictions into serving-policy proxies. They estimate prefetch opportunity, dependency-safe scheduling behavior, and EPLB-style placement improvement.

sampling parameters, or model weights. We nevertheless report exact-output match as a measurement rather than a proof obligation because the full run uses independently restarted vLLM shards; batch execution is not bitwise stable across all shard and batch-size configurations. Exact output match is 100% for batch size 1 and 85.2–87.0% for batch sizes 2–8.

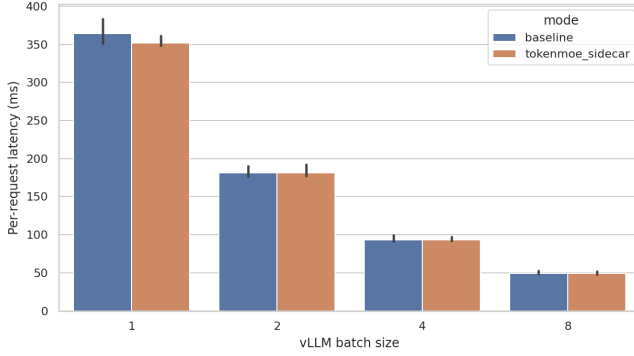
6.5 Reproducibility and Failures (Q4)

All experiment attempts are preserved. The environment logs include the initial unresolved vLLM install, the corrected PyTorch/vLLM installation, the Transformers compatibility failure, and the final verified stack. The vLLM logs include successful probe runs, a long-process cleanup hang after 60 baseline requests, two OOM failures caused by external GPU occupation, and the final resumable sharded run. These failures are not hidden because they affect how the final experiment should be interpreted: the reported vLLM latency is a real shared-machine measurement with explicit resource provenance, not an isolated benchmark.

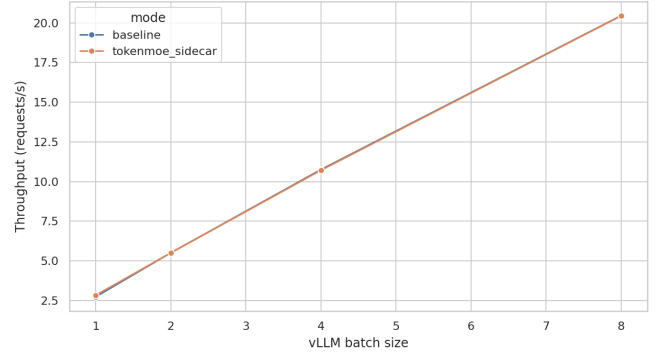
7 Related Work

LLM serving. vLLM [1] popularized paged attention and high-throughput request scheduling for LLM serving. SGLang [2] exposes structured generation programs and runtime optimizations for agent-like workloads. TokenMoE is complementary: it targets early expert working-set prediction for MoE models and keeps the model router authoritative.

MoE systems. Expert-parallel and expert-placement systems optimize how sparse experts are placed and balanced across accelerators. EPLB-style policies use observed token load to rebalance expert placement. TokenMoE adds



(a) Latency by batch size



(b) Throughput by batch size

Figure 5. Real vLLM run on the requested Qwen2.5-7B-Instruct model. The target is dense, so the experiment measures serving integration and sidecar overhead rather than routed-expert speedup.

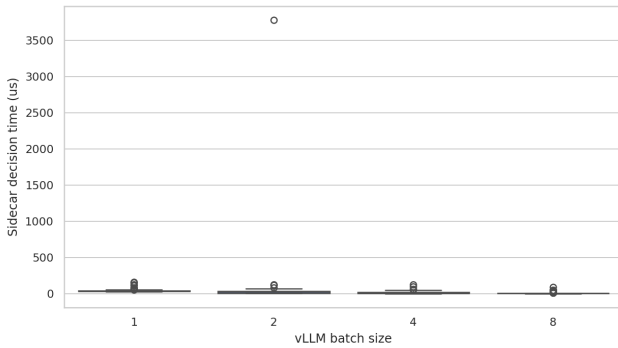


Figure 6. TokenMoE sidecar decision time in the dense Qwen2.5-7B-Instruct fallback path. The sidecar records metadata hashes, fallback keys, and explicit fallback reason for every request.

a request-level prior from agent metadata, which can be combined with observed load rather than replacing it.

Cache and agent serving. Recent cache-centric systems exploit prefix reuse, position-independent reuse, or KV movement to reduce context cost. Agent schedulers exploit graph structure and dependencies. TokenMoE studies a different resource: routed expert demand. The same agent graph metadata that helps scheduling can also predict expert working sets before decode.

Routing predictability. Prior work observes locality and specialization in MoE routers. TokenMoE turns this observation into a serving-side artifact: a hierarchical online store, confidence estimates, trace format, and fallback behavior that can be tested on both routed and dense models.

8 Conclusion

TokenMoE studies a practical question for MoE LLM serving: can agent metadata predict expert working sets early enough

to guide systems decisions? Real routed traces show that hierarchical agent metadata improves top-4 expert prediction over a global baseline, and replay simulators expose prefetch and placement opportunities. The requested Qwen2.5-7B-Instruct vLLM experiment establishes the other half of the contract: dense models must take an explicit no-router fallback rather than produce misleading MoE traces. The result is a trace-first, reproducible prototype that separates opportunity, safety boundary, and deployment mechanics.

References

- [1] Woosuk Kwon, Zhuohan Li, Siyuan Zhuang, Ying Sheng, Lianmin Zheng, Cody Hao Yu, Joseph E. Gonzalez, Hao Zhang, and Ion Stoica. 2023. Efficient Memory Management for Large Language Model Serving with PagedAttention. In *Proceedings of the ACM SIGOPS Symposium on Operating Systems Principles*.
- [2] Lianmin Zheng, Liangsheng Yin, Zhiqiang Xie, Chuyue Huang, Jeff Sun, Cody Hao Yu, Shiyi Cao, Christos Kozyrakis, Ion Stoica, Joseph E. Gonzalez, Clark Barrett, and Ying Sheng. 2023. SGLang: Efficient Execution of Structured Language Model Programs. arXiv:2312.07104